

Agent Verification System

Automation Case Study

A design proposal for verifying new agents at scale

Author: Himanshi Kohli

Role applied for: Automation Engineer

Date of Submission: May 8th, 2026

Table of Contents

1. Executive Summary	2
2. Solution Overview	3
3. Sequence Diagram	5
4. Data Flow Diagram (DFD)	6
4.1 N8N Workflows	7
4.2 Playwright – Workers (Dataflow)	9
5. Technology Choices and Reasoning.....	10
5.1 Considered Alternatives.....	11
6. Design Questions.....	12
6.1 How do we mitigate failures from UI changes in JoinReal.com or the DRE websites?	13
6.2 How do we surface failures for human review?	14
6.3 What is the deployment strategy for always-on operation?	15
7. Assumptions	17
8. Limitations	18
9. Open Questions	18
Appendix A — Failure Router Mechanism	19
Appendix B — Selector Registry Schema.....	21

1. Executive Summary

When a new Real agent completes onboarding, three conditions must be verified within minutes. First, the agent must be active in the CRM. Second, the agent must appear correctly on the public onereal.com directory under the correct licensing state. Third, the license expiration date stored in the CRM must match the expiration date listed on the corresponding state Department of Real Estate (DRE) website.

This document proposes an **event-driven automation**. A **CRM webhook** triggers a short pipeline that runs four steps in order: pull agent data through an API call, check the JoinReal directory, check the state DRE site, and compare the two. If everything matches, the system marks the agent as verified in the CRM. If anything fails, a single **Failure Router** decides whether to retry, file a **Jira** ticket for human review, or page on-call.

A **24-hour cron job** runs the same flow against test agents whose data we already know. It catches website changes before any real agent encounters them.

The stack is deliberately simple and well-understood: **n8n** for orchestration, **Playwright** for browser automation, **Postgres** for state, **Redis** for queueing, **S3** for screenshots, Jira for review tickets, and **NocoDB** as the database editor. AI-driven recovery using Stagehand or browser-use is planned as Phase 2 — added on top of the deterministic foundation, not replacing it.

2. Solution Overview

The system is organised into five clear sections. Each section has a single, well-defined responsibility, and the sections talk to each other through known boundaries (a webhook, a queue, a database).

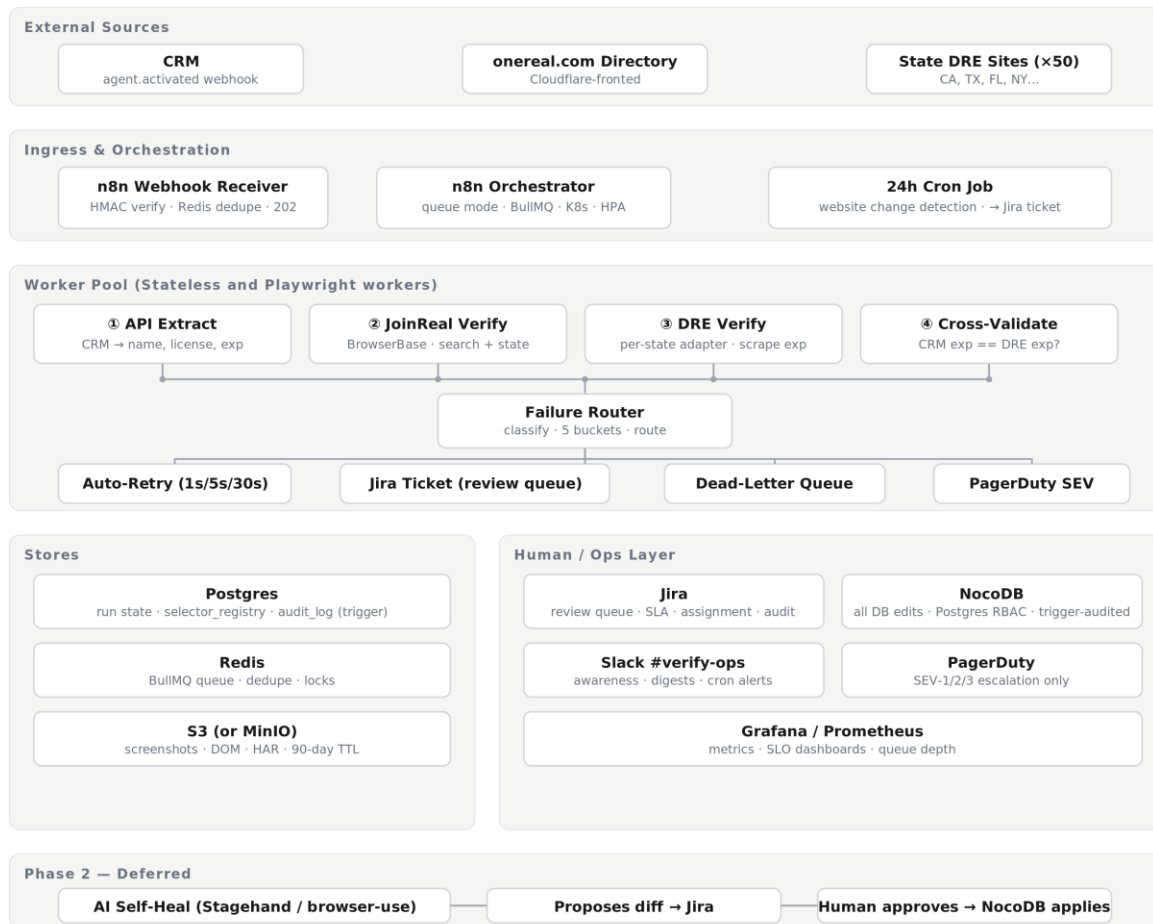


Figure 1 — System Architecture

External Sources

The CRM emits an *agent.activated* event when onboarding is complete. The two websites — onereal.com (the public agent directory) and the state DRE sites — are the targets the system needs to read from.

Ingress and Orchestration

An **n8n** webhook receiver catches the event, verifies its signature, and pushes it onto a queue. The n8n orchestrator pulls jobs from the queue and runs them step by step. A separate **24-hour cron job** emits a "test event" **once a day** to make sure the same UI path still works.

Worker Pool

Four small, stateless workers do the actual work in order:

- **API Extract Worker** calls the CRM and pulls the agent's name, state, license number, and expiration date (on n8n).
- **JoinReal Verify Worker** uses Playwright (a browser automation library) to open onereal.com, search by name, click the profile, and confirm the agent is listed under the right state.
- **DRE Verify Worker** does the same against the state's DRE site. It searches by license number, opens the listing, and reads the official expiration date.
- **Cross-Validate Worker** compares the CRM expiration with the DRE expiration. If they match, it marks the agent as verified in the CRM. (on n8n)

Stores

Postgres holds the run state, the selector registry, the audit log, and feature flags. **Redis** acts as the queue and dedupe layer. **S3** holds screenshots and page snapshots — useful for debugging and audit.

Human and Ops Layer

When a worker fails, the **Failure Router** classifies the failure and decides what to do. In most cases, it creates a **Jira** ticket. Ops engineers review the ticket and fix the issue using **NocoDB**, a web-based table editor that uses **Postgres roles** for access control. **Slack** and **PagerDuty** handle alerts. **Grafana** and **Prometheus** monitor system health.

3. Sequence Diagram

A full verification — from CRM event to writeback — flows through 21 numbered steps. The sequence diagram below shows every interaction between components in the order they happen.

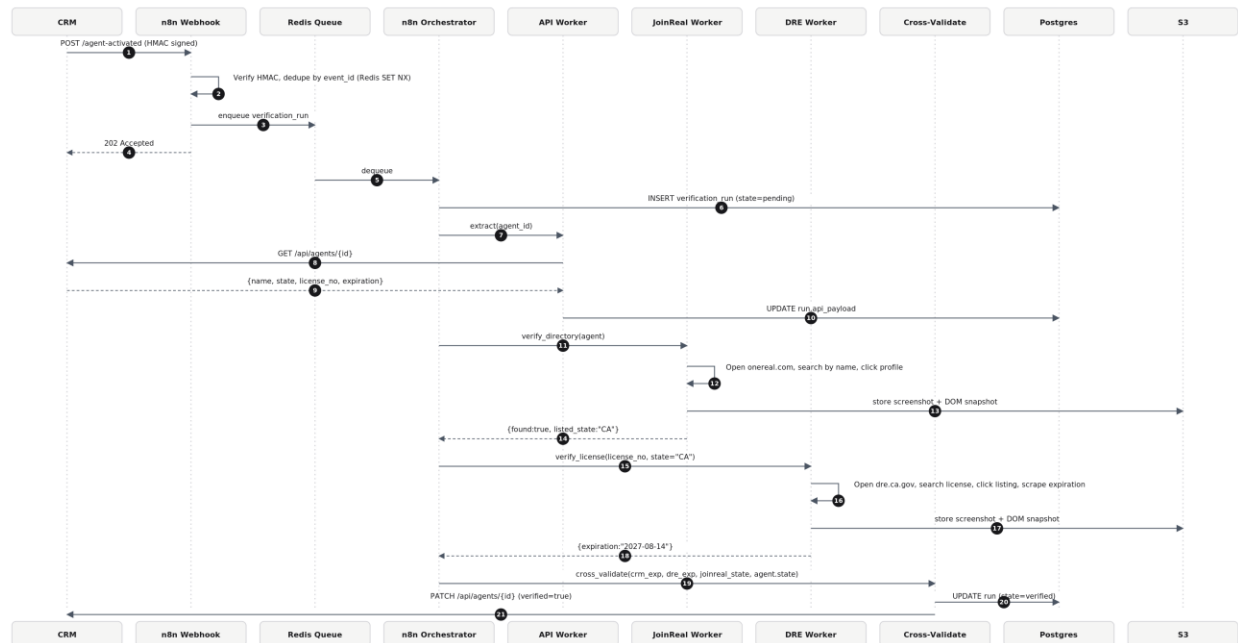


Figure 2 — Happy Path Sequence Diagram

The flow can be summarized in four stages:

- **Steps 1–4 (Ingress).** The CRM sends the event. The webhook verifies its signature, deduplicates by event ID using Redis, queues the work, and returns a quick "202 Accepted" so the CRM is not kept waiting.
- **Steps 5–10 (Extract).** The orchestrator dequeues the job, records it in Postgres, and asks the API Extract Worker to call the CRM. The worker stores the agent's metadata in Postgres.
- **Steps 11–18 (Verify).** The JoinReal Worker opens onereal.com, searches, opens the profile, stores screenshots in S3, and returns the listed state. The DRE Worker does the same against dre.ca.gov and returns the official expiration date.
- **Steps 19–21 (Cross-Validate and Writeback).** The Cross-Validate Worker compares the values, marks the run as verified in Postgres, and writes the result back to the CRM.

A normal run completes in about 30 to 45 seconds. The target is 95% under 90 seconds, 99% under 5 minutes.

4. Data Flow Diagram (DFD)

The data flow diagram below shows the same pipeline from a different angle: which data moves between which components, and what each piece of data contains. External services that the workers talk to (CRM, onereal.com, DRE sites) are placed on the left, next to their corresponding worker. The pipeline runs down the centre. The failure branch is on the right. Supporting components (Postgres, S3, Grafana, NocoDB, Ops Engineer) are at the bottom.

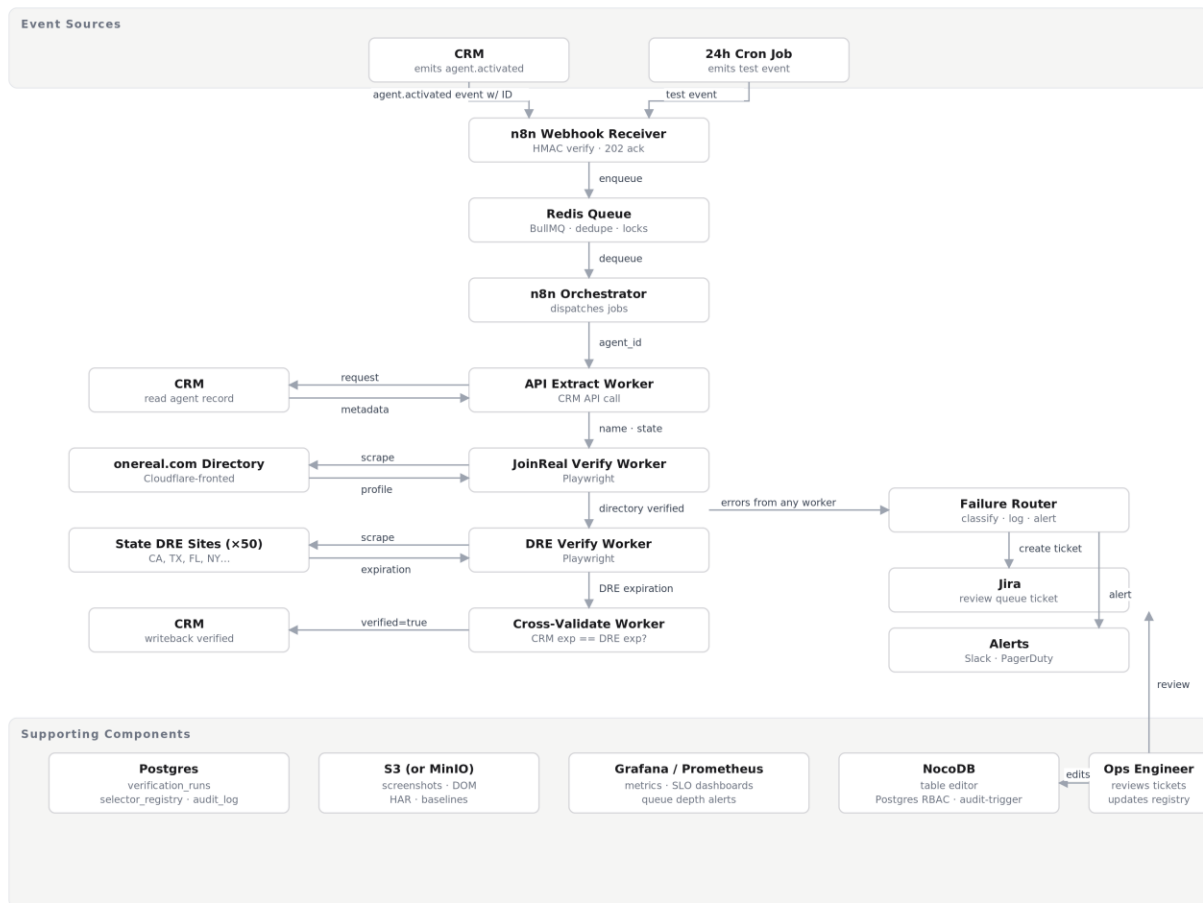


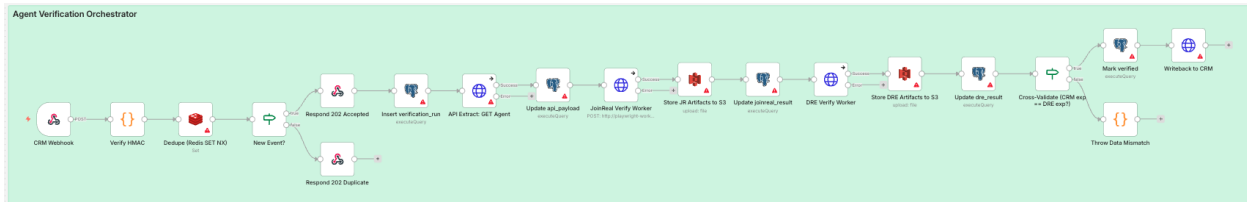
Figure 3 — Data Flow Diagram

Three things are worth noticing. First, both the **CRM** and the **24-hour cron job** feed events into **the same n8n webhook**. The webhook does not care whether the event is real or a test; both go through the same pipeline. Second, all **four workers share one error path** — any failure goes to the **Failure Router**, which decides what to do next. Third, the **Ops Engineer** interacts with the system through two surfaces only: **Jira** (to review tickets) and **NocoDB** (to update the database).

4.1 N8N Workflows

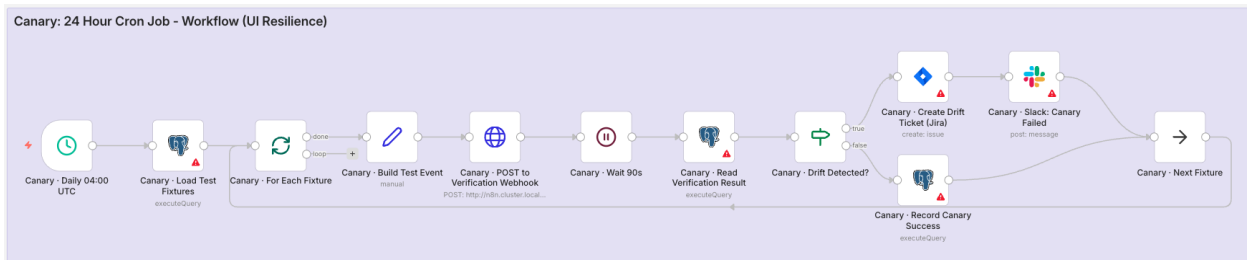
1. N8N Orchestrator – Agent Verification

Triggered when the CRM emits an agent-activated event, this workflow validates the webhook, deduplicates the event, retrieves agent metadata, and runs browser-based verification against JoinReal and the state DRE website. The results are cross-validated and written back to the CRM, while screenshots and audit data are stored for traceability.



2. Canary / Drift Detection Workflow

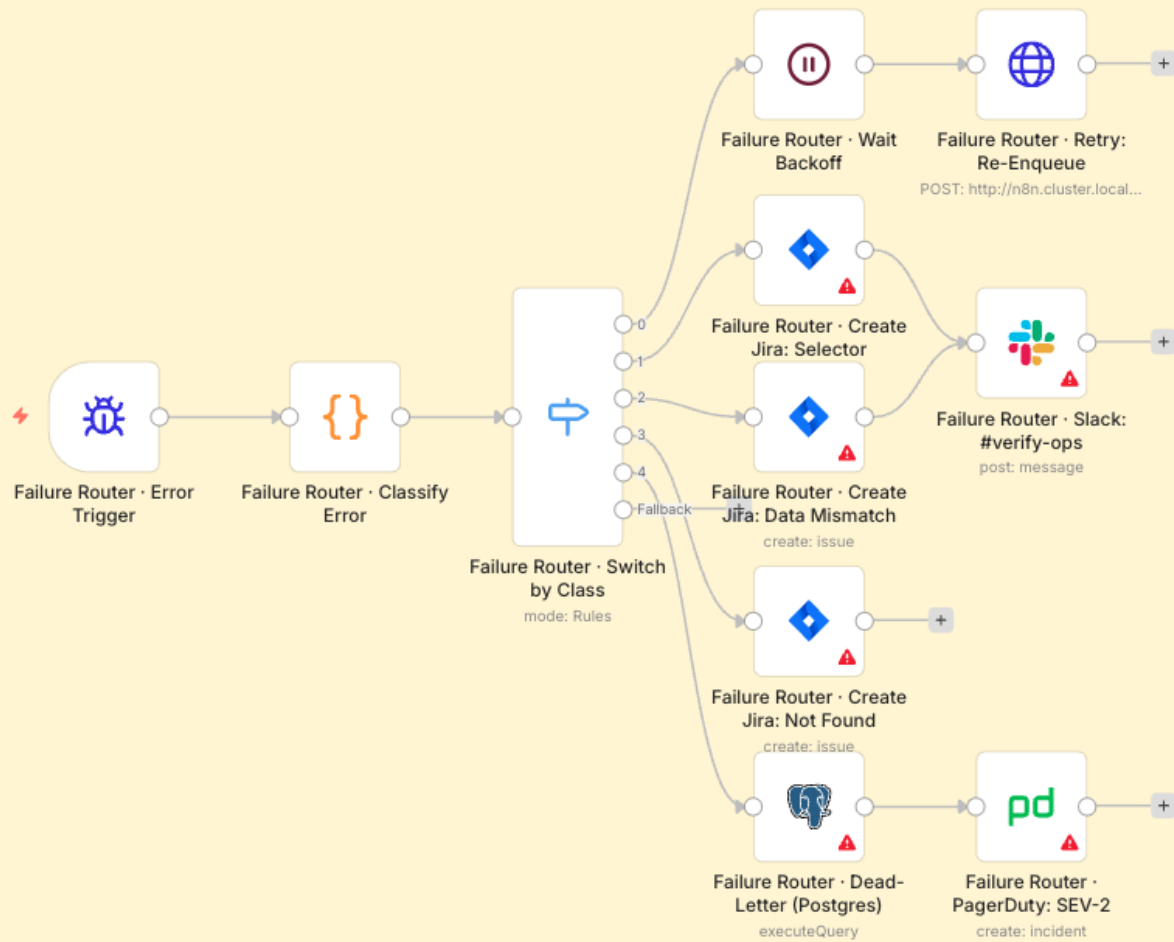
A scheduled workflow runs every 24 hours using synthetic test agents with known expected results. It replays the same verification flow end-to-end to detect UI changes, selector drift, or unexpected behavior before production agents are impacted.



3. Failure Router

All workflow failures are routed through a centralized error-handling pipeline that classifies issues into categories such as transient, selector, mismatch, or poison failures. Based on the failure type, the system automatically retries, creates Jira tickets, sends Slack alerts, or escalates critical incidents through PagerDuty.

Failure Router - Workflow



4.2 Playwright – Workers (Dataflow)

1. Playwright Worker 1 — JoinReal Verification Worker

1. Receive agent details from orchestrator (n8n)
2. Load selectors from selector registry (in postgres)
3. Launch Playwright browser session
4. Navigate to JoinReal directory
5. Search agent by name
6. Open matching agent profile
7. Verify listed state matches CRM state
8. Capture screenshots/artifacts
9. Upload artifacts to S3
10. Return verification result to orchestrator
11. Handle selector failures/timeouts gracefully
12. Close browser session

2. Playwright Worker 2 — DRE Verification Worker

1. Receive license number and state from orchestrator (n8n)
2. Select state-specific DRE adapter
3. Load selectors for target DRE site
4. Launch Playwright browser session
5. Navigate to state DRE website
6. Search using license number
7. Open matching license record
8. Extract license expiration date
9. Capture screenshots/artifacts
10. Upload artifacts to S3
11. Return expiration data to orchestrator
12. Handle CAPTCHA, selector, or timeout failures
13. Close browser session

5. Technology Choices and Reasoning

Component	Chosen	Why
Webhook + Orchestrator	n8n (in queue mode)	Visual workflow editor, native error handling, runs on Kubernetes. Open-source. Easy to operate.
Queue	Redis (BullMQ)	Comes with n8n queue mode. No new infrastructure to add.
Browser automation	Playwright	Best modern API. Auto-wait, multi-engine support, stable selectors, mature ecosystem.
Browser hosting	Self-hosted with BrowserBase as fallback	Self-host is cheap at steady volume. BrowserBase provides residential IPs and fingerprint diversity for Cloudflare-protected sites like oneread.com.
Database	Postgres	One database for run state, selector registry, audit log, and feature flags. JSONB columns keep payloads flexible.
Object storage	S3 (or MinIO on-prem)	Cheap, durable, supports signed URLs for review UI, lifecycle rules for retention.
Review queue	Jira	Already used at most companies. Comes with SLA tracking, assignment, and audit history for free.
Database editor	NocoDB	Web UI for safely editing tables. Uses Postgres roles for access control. Self-hostable.
Alerting	Slack + PagerDuty	Usual Industry standard. Slack for awareness, PagerDuty for action.
Monitoring	Grafana, Prometheus, Loki	Self-hostable. One observability stack.
Deployment	Docker, Helm, Kubernetes	Standard. Auto-scales workers based on queue depth.

5.1 Considered Alternatives

Three deliberate non-choices are worth calling out.

- **Temporal instead of n8n.** Temporal is a more powerful orchestration tool with stronger durable-execution guarantees. It is also harder to operate and harder for non-engineers to read. n8n is enough for the volume in scope. The trigger to switch to Temporal would be workflows running over an hour, fan-out larger than 1,000 child runs, or a need for strict at-least-once-with-exactly-one-side-effect semantics.

- **AI-only self-healing instead of a deterministic foundation.** Stagehand and browser-use are exciting but new. Building the foundation on them in Phase 1 would mean depending on emerging tooling for the load-bearing layer. Phase 1 is deterministic; Phase 2 adds AI on top of it as a fallback when the deterministic layer cannot cope. This is described in section 6.1.
- **Retool for the review queue.** Retool is a fast way to build internal tools, but it is one more SaaS contract and one more system for the on-call team to learn. Most companies already run Jira. Using Jira saves on tooling, gets SLA and audit features for free, and keeps the on-call experience consistent.

6. Design Questions

6.1 How do we mitigate failures from UI changes in JoinReal.com or the DRE websites?

UI changes are the failure mode the system is designed to survive. We use four layers of defence.

Layer 1 — Multi-strategy locator ladder

Every interactive element on a page (a button, an input field) is found using up to four strategies, in priority order:

1. A data-testid attribute (most stable; developers usually do not rename it).
2. An ARIA role with an accessible label (stable because it is tied to accessibility, not styling).
3. Visible text (reasonably stable; it is a product decision, not a code detail).
4. A CSS selector (least stable; falls back here only as a last resort).

The worker tries strategy 1 first. If it fails, strategy 2 is tried, and so on. A run only "fails-by-selector" when all four strategies fail. This means the website would have to remove the testid AND break the ARIA label AND change the visible text AND restructure the HTML — all at once — to take down a single element. That almost never happens.

Layer 2 — Versioned selector registry

The four strategies for each element are not hardcoded in the worker code. They live as rows in a Postgres table called `selector_registry`. Each row stores a JSON array of strategies, a version number, and an active flag. When a website changes, ops engineers update the row through NocoDB; the running workers pick up the new strategies on their next run. No code change. No deployment. Each change is versioned, so rollback is instant — flip the new row's active flag to false, and workers go back to the previous version.

A Postgres trigger fires on every change to `selector_registry` and writes the change to an audit log automatically. That means the audit invariant cannot be bypassed, no matter which tool made the change. The full schema is in Appendix B.

Layer 3 — 24-hour cron job for website-change detection

A scheduled job runs once a day. It emits a test event into the same n8n webhook that real CRM events use. The pipeline runs end-to-end against fake test agents whose data we already know. The result is compared against a saved baseline:

- Field values are compared exactly (expected name, expected expiration date).
- Page structure is compared against the last-known-good DOM with some tolerance.
- Critical-page screenshots are compared against a baseline using a perceptual diff.

If anything has drifted, a Jira ticket is filed with the screenshot diff highlighted, before any real agent encounters the broken state. The cron runs at a quiet hour (4 a.m. UTC) so it does not compete with real traffic.

Layer 4 — Per-target feature flags

Each external site has a feature flag (for example, `flag.dre.ca.enabled`). If California breaks at 3 a.m. and we cannot fix the selectors fast enough, an ops engineer flips the flag off. New California verifications go straight to the review queue with a "DRE down" tag instead of crash-looping. The system keeps running for the other 49 states. When the canary turns green again, the flag is flipped back on.

Phase 2 — AI-driven self-healing

When the locator ladder runs out — all four strategies fail on a real run — the worker can hand off to a Stagehand or browser-use agent given the goal in plain English: "On this page, find the input where I would type an agent's name to search." If the AI agent finds a working locator, it posts the proposed selector as a comment on the Jira ticket. A human reviews and approves it. Only then is the new selector applied through NocoDB and added as the new top strategy in the registry. The schema enforces this: a check constraint refuses to mark any AI-sourced row as active without an approver. AI proposes; humans approve.

6.2 How do we surface failures for human review?

Failures flow through one component: the Failure Router. It does three things — classifies the error, decides what to do with it, and routes it to the right place. The full classification is in Appendix A. The short version is below.

Failure Class	What it Means	Routing
Transient	Network 5xx, timeout, navigation error	Auto-retry up to 3 times with exponential backoff (1s, 5s, 30s). No human is bothered.
Selector	All four locator strategies failed	Phase 1: Jira ticket and Slack alert. Phase 2: try AI self-heal first.
Data Mismatch	CRM says one expiration, DRE says another	Jira ticket (HIGH priority). Often signals a real compliance issue.
Not Found	Search returned 0 results	Jira ticket (MEDIUM priority). May be a propagation race or stale data.
Poison	Uncaught exception or run timeout > 5 minutes	Dead-letter queue and PagerDuty page.

Each Jira ticket carries everything an on-call engineer needs to act in under five minutes: agent name, state, and license number; failure class and human-readable reason; a screenshot at the moment of failure (linked from S3); a snapshot of the page; the strategies that were tried; and a direct link into NocoDB to fix the row if the issue is a selector problem. Tomorrow's cron run verifies the fix.

Alerts are tiered to avoid noise:

- A single failure produces a Jira ticket only — no Slack ping. The queue is the system of record.
- If the review queue grows by 10 or more in five minutes, a Slack message goes to #verify-ops.
- If the same selector fails across many runs in five minutes, the system fires a Slack alert and automatically disables the affected adapter via feature flag.
- If review wait time grows beyond four hours, PagerDuty escalates to SEV-3 (business hours).
- If the CRM webhook is unreachable for more than 10 minutes, PagerDuty escalates to SEV-1.

6.3 What is the deployment strategy for always-on operation?

The system runs on Kubernetes, deployed through Helm charts and GitHub Actions. The design choices that keep it always-on are:

Stateless webhook receivers

The webhook receiver verifies the event, deduplicates it through Redis, queues it, and returns 202 Accepted. Losing one webhook pod is invisible to the CRM because pods are interchangeable and a load balancer routes around failures. n8n queue mode separates webhook ingestion from processing, so a stuck workflow can never block a new event from being accepted.

Redis as the durability boundary

Once a webhook returns 202, the event lives in Redis. If every worker pod dies, the queue persists. New workers drain it on startup. Nothing is lost.

Autoscaling on queue depth

Kubernetes autoscales workers based on Redis queue length, not CPU. A spike of 1,000 onboardings in an hour scales workers up automatically; the queue acts as a shock absorber that smooths bursty input into steady-rate output.

Per-target feature flags

As described in §6.1, each external website is gated by a feature flag. One broken DRE does not poison the system. Failed runs go to review, not into a retry storm.

Multi-region active-passive

Postgres replicates asynchronously to a disaster-recovery region. Redis replicates with Sentinel. RPO is roughly one minute; RTO is roughly 15 minutes. Active-active is not justified at the current volume but can be added later if needed.

Idempotent design

The CRM may retry the same event. The receiver deduplicates by event_id using Redis SET NX. The same event delivered five times produces exactly one verification run. A unique constraint on (agent_id, crm_event_id) in the verification_run table is a second line of defence at the database layer.

Service Level Objectives

Indicator	Target
Webhook ingestion availability	99.95% per month
End-to-end p95 (event to verified)	Under 90 seconds
End-to-end p99	Under 5 minutes

Indicator	Target
Auto-resolved (no human intervention)	At least 97%
Review queue p95 wait time	Under 4 hours, business hours
Daily cron success rate per active state	At least 99% per day

Deployment workflow

Container images are promoted by tag through the pipeline: dev → staging → prod-canary (5%) → prod (100%). n8n workflow definitions are version-controlled as JSON in the repository and synced to the running instance through n8n's API at deploy time. Rollback is a single Helm command, or a SQL update to the selector_registry that flips the new version's active flag back to false.

7. Assumptions

- The CRM emits an agent.activated webhook secured with an HMAC signature. If only database CDC is available, a Debezium-to-Kafka ingestion layer can be introduced.
- The CRM exposes a read API that returns the agent's name, state, license number, expiration date, and active status.
- The CRM supports a writeback API to update verified=true and store a verified_at timestamp.
- JoinReal.com is publicly accessible, and the search-and-profile flow is stable enough to support deterministic browser automation.
- State DRE websites allow public license lookup without authentication. States requiring login or paid access will require dedicated credentialed adapters in a later phase.
- Phase 1 is designed for approximately 5,000 verification events per day with up to 50 concurrent runs, which is within the operational capacity of n8n queue mode.
- Existing infrastructure such as Kubernetes, Postgres, Slack, and PagerDuty is already available. If not, the system can alternatively be deployed using ECS Fargate.
- The onboarding event is emitted only after the agent is fully available in downstream systems such as JoinReal.com. If propagation delays exist, a configurable wait period can be introduced before verification begins.

8. Limitations

- Full state coverage will take time. Until adapters are built for all 50 states, agents from unsupported states will bypass DRE verification and be routed to the review queue for manual handling.
- Cloudflare or future anti-bot protections may introduce CAPTCHA or automation-resistant behavior. This can be mitigated through hardened browser infrastructure and, if required, internal B2B data-sharing arrangements.
- The system is asynchronous and event-driven. Verification is designed around an SLO target (p95 under 90 seconds), not synchronous real-time responses.
- AI-assisted selector recovery in Phase 2 introduces controlled non-determinism. All AI-generated selector updates require human approval before activation.
- DRE rate limits and Terms of Service vary by state and are not consistently documented. Each state adapter rollout should include legal review, rate limiting, and polite automation controls such as request throttling and identifiable User-Agent headers.

9. Open Questions

These are the things I would want answered before writing the first line of code, in priority order:

5. **CRM identity and webhook contract.** Salesforce, an internal platform, or something else? What is the exact event payload, signing scheme, and retry policy?
6. **CRM API SLA and rate limits.** How fast does the new agent record become readable after agent.activated? Is there a rate-limit ceiling we would hit at peak?
7. **Existing automation footprint.** Does Real already run n8n, Temporal, Airflow, or Workato anywhere? Standardising on the existing tool wins over introducing a new one.
8. **DRE legal and ToS posture.** Is there an internal legal opinion on automated DRE access? Are any states explicitly off-limits?
9. **Volume forecast.** What is the actual peak weekly onboarding rate? Concurrent peak? When does the RE/MAX merger start hitting this pipeline?
10. **PII scope on screenshots.** Do screenshots count as PII storage under Real's data classification? Affects S3 retention and access controls.
11. **SLA expectation.** Is "verified within 90 seconds" right, or does the business want minutes / same-day / next-day? This shifts cost dramatically.
12. **Mismatch escalation.** When CRM expiration $\neq$ DRE expiration, who is the source of truth? Does the system auto-correct CRM, or always flag for human review?
13. **International readiness window.** After RE/MAX integration, when does Canadian onboarding start hitting this pipeline? Provincial regulators differ from DREs.
14. **Existing dashboarding.** Does Real already have an internal ops portal that should host the review queue, instead of standalone Jira projects?

Appendix A — Failure Router Mechanism

The Failure Router is the single component that decides what happens when any of the four workers fails. It is implemented as an n8n error workflow that receives errors from the worker steps. It does three things in order: classify, route, and decide on noise level.

A.1 Inputs

The router receives two inputs:

15. **The error payload.** Includes which worker failed, which step, the error type, the HTTP status code if relevant, the selector that was being attempted, the strategies that were tried, and the retry count.
16. **The verification_run row from Postgres.** Includes agent metadata, prior step results, and S3 URLs for screenshots already captured.

A.2 Classification (5 buckets)

A simple rule-based classifier maps the error to one of five classes. No machine learning, no fancy logic — just a switch statement on the error type and retry count.

Class	Trigger	Routing
Transient	Network 5xx, timeout, navigation error	Auto-retry with exponential backoff (1s, 5s, 30s). After three attempts, promote to selector or poison and escalate.
Selector	Locator ladder exhausted on a real run (not a canary)	Phase 1: create Jira ticket, ping Slack #verify-ops. Phase 2: try Stagehand or browser-use agent first; fall through to Jira if AI also fails.
Data Mismatch	Cross-Validate found CRM ≠ DRE	Create Jira ticket (HIGH priority). Tag the agent in CRM. Often signals a real compliance issue (e.g., agent's license actually expired).
Not Found	Search returned 0 results on JoinReal or DRE	Create Jira ticket (MEDIUM priority). Tag the agent.
Poison	Uncaught exception, run timeout exceeding 5 minutes, OOM crash	Move the message to a dead-letter queue. Page on-call via PagerDuty.

A.3 Aggregate Alerts

Some signals only emerge across many failures. A separate alert rule on top of the router watches for patterns and reacts:

- If 10 or more new tickets are created in five minutes, ping Slack #verify-ops.

- If the same selector fails across many runs in five minutes, the alert rule both pings Slack and automatically disables the affected adapter through its feature flag. The system protects itself.
- If review queue p95 wait time exceeds four hours during business hours, escalate to PagerDuty SEV-3.
- If the CRM webhook is unreachable for more than 10 minutes, escalate to SEV-1.

A.4 Ticket Payload

Every Jira ticket the router creates includes the same set of fields, so on-call engineers always know where to look:

- Agent ID, name, state, and license number.
- Run ID and CRM event ID.
- Failure class and a human-readable reason.
- The selector that failed and the strategies that were tried.
- Screenshot URL (S3 signed link) for the moment of failure.
- DOM snapshot URL (S3 signed link).
- A direct link into NocoDB filtered to the selector registry rows that may need updating.
- A reference to the underlying audit_log row, for compliance traceability.

Appendix B — Selector Registry Schema

The full SQL schema is in `selector_registry_schema.sql`, delivered alongside this document. The core ideas are summarised below.

B.1 Table Layout

Each row represents one element on one website's one workflow at one version. Columns include:

- *target* — the website, e.g., `joinreal`, `dre.ca`, `dre.tx`.
- *flow* — the workflow on that site, e.g., `search_by_name`, `license_lookup`.
- *step* — the element within that flow, e.g., `name_input`, `submit_button`.
- *strategies* — a JSONB array of locator strategies in priority order: `testid`, then ARIA role, then visible text, then CSS.
- *version*, *active* — the version number and active flag. The unique index on (`target`, `flow`, `step`, `version`) prevents accidental duplicates within a version.
- *source*, *created_by*, *created_at*, *approved_by*, *approved_at* — provenance fields. `source` is one of `human`, `ai-self-heal`, or `migration`.
- *jira_ref*, *notes* — cross-references to the Jira ticket that prompted the change and free-text operator notes.

B.2 Sample Strategy Array

A typical row's `strategies` field looks like this:

```
[ { "kind": "testid", "value": "agent-search-name", }, { "kind": "role", "role": "textbox", "name": "Find Agent by Name", }, { "kind": "text", "value": "Find Agent by Name", }, { "kind": "css", "value": "input[placeholder*='agent' i]" } ]
```

B.3 Audit Trigger

A Postgres trigger fires on every `INSERT`, `UPDATE`, or `DELETE` on the `selector_registry` table. It writes a row to the `audit_log` table with the actor, the action, the previous and new values, and the Jira reference. The trigger is declared `SECURITY DEFINER` so callers (e.g., `ops_editor` in NocoDB) can write to `selector_registry` without needing direct `INSERT` permission on `audit_log`. This means the audit invariant cannot be bypassed by any client.

B.4 Postgres Roles

Four roles control access at the database level:

- *app_worker* — read-only on `selector_registry`. Used by the verification workers.
- *ops_editor* — read, insert, and update on `selector_registry`. Used by NocoDB for ops engineers.
- *ops_readonly* — read-only across the system. Used for dashboards and audit reviews.

- *ai_proposer* — insert-only on `selector_registry`. Used by Phase 2 AI self-heal. AI-proposed rows cannot self-activate; a check constraint enforces that any row with `source=ai-self-heal` and `active=true` must have an approver.

B.5 Worker Read Pattern

Workers read selectors using a defensive query that always picks the highest-version active row:

```
SELECT strategies FROM selector_registry WHERE target = $1 AND flow = $2 AND step = $3  
AND active = TRUE ORDER BY version DESC LIMIT 1;
```

A partial index on `(target, flow, step, version DESC) WHERE active = TRUE` makes this lookup $O(1)$. The defensive `ORDER BY` also means the brief window during a NocoDB edit when both old and new rows are momentarily active is harmless — workers always pick the higher version.

